claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_87b22fa4-6ac\agent-adcb8122a9d4230b5.jsonl`
- SHA-256: `a93481228a71aa1c7e1daede0b333d8b37233175618620a3b23d110a6bbdc283`
- Source modified: `2026-06-21T10:48:01+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_87b22fa4-6ac`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T10:45:15.846Z)

Review the COMPUTE_DECOUPLED_SERVING **M6** change just committed on branch feat/serving-m6-cloudrun-dispatch (HEAD). Inspect with: git diff origin/master...HEAD , and read the full files:
- backend/compute/base.py (ComputeBackend ABC + ComputeJobSpec + ComputeResult)
- backend/compute/cloud_run.py (CloudRunCompute: dispatch a Cloud Run Job via injectable CloudRunJobClient, then bucket-poll a done-marker via execution_policy.poll_until + the storage layer; GoogleCloudRunJobClient = real client, lazy google-cloud-run)
- backend/compute/__init__.py (get_compute_backend(config) ? backend only for compute_target=cloud_run, else None = run inline)
- backend/worker/__main__.py (cmd_infer: dispatch branch for cloud_run + _dispatch_remote; --done-uri + _write_done_marker on success)
- deploy/cloudrun/Dockerfile + README.md (the L4 image)
- tests/test_compute_backend.py
CONTRACT: DEFAULT-OFF ? only compute_target=cloud_run dispatches; ""/local_gpu/cpu_mock run the worker's EXISTING in-process path BYTE-IDENTICALLY. The dispatched container must run INLINE (never re-dispatch) via forced compute_target=local_gpu+native. Completion = a bucket done-marker (not the Cloud Run API status), D16. The real client is owner-verified on M7 (CI uses a fake). Report ONLY real, grounded issues with file:line. Empty findings array if a lens finds nothing real ? do not invent nits.

LENS = TEST RIGOR / FALSE-CONFIDENCE. (1) The fake _FakeRunClient writes the marker SYNCHRONOUSLY during run_job, so when CloudRunCompute then bucket-polls, the marker already exists ? poll_until's FIRST check() returns it and the loop never actually waits/loops. Does any test exercise poll_until's wait path (marker appears after ?1 None poll)? If not, the polling loop is effectively untested here (only execution_policy's own tests cover it) ? is that a meaningful gap for the M6 'bucket-poll' claim? (2) test_cloud_run_dispatch_polls_and_returns_result ? does it prove the dispatcher reads the RIGHT output (video_id-keyed path) vs coincidence? (3) test_worker_writes_done_marker_on_success ? is it non-vacuous (segment_count>=1 guards it)? (4) The Dockerfile guard is string-match only ? adequate? (5) Any assertion that would pass even if the dispatch shim were broken? Find vacuous/weak coverage.

2. assistant (2026-06-21T10:45:18.546Z)

I'll review the M6 change. Let me start by pulling the latest and inspecting the diff and the relevant files.

3. user (2026-06-21T10:45:20.453Z)

2266e91 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
---BRANCH---
feat/serving-m6-cloudrun-dispatch
---DIFF STAT---
 backend/compute/__init__.py | 61 ++++++++++++

```
 backend/compute/base.py       |  52 ++++++++++
 backend/compute/cloud_run.py  | 141 ++++++++++++++++++++++++++
 backend/worker/__main__.py    |  51 ++++++++++
 deploy/cloudrun/Dockerfile    |  61 ++++++++++++
 deploy/cloudrun/README.md     |  59 ++++++++++++
 tests/test_compute_backend.py | 222 ++++++++++++++++++++++++++++++++++++++++++
 7 files changed, 647 insertions(+)
```

4. assistant (2026-06-21T10:45:29.952Z)

Let me read all the files in parallel.

5. user (2026-06-21T10:45:30.908Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure). ``video_id`` is known once
the job's
37          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
38
39          returncode: int
40          outputs_uri: str = ""
41          video_id: str = ""
```

```
42          report: dict = field(default_factory=dict)
43
44
45      class ComputeBackend(ABC):
46          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
47          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
48          execution identically."""
49
50          @abstractmethod
51          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
52              """Run ``spec`` to completion and return its ``ComputeResult``."""
53
```

6. user (2026-06-21T10:45:31.398Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19      owner-verified on the M7 real run.
20      """
21
22      from __future__ import annotations
23
24      import json
25      import logging
26      import uuid
27      from abc import ABC, abstractmethod
28      from typing import Any, List, Optional
29
30      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
31      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
32      from backend.storage import fetch, parse_uri
33      from backend.storage import get_storage
34
35      LOG = logging.getLogger("compute.cloud_run")
36
37      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
```

```
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""

        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(

container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
            )
            op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
            return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
                     project: Optional[str] = None, storage_config: Optional[dict] = None,
                     policy: ExecutionPolicy = DEFAULT_POLICY,
                     token: Optional[str] = None,
                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
            self._client = run_client
            self._job_name = job_name
```

```
90              self._region = region
91              self._project = project
92              self._storage_config = storage_config or {}
93              self._policy = policy
94              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
95              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
96              self._token = token
97              self._container_overrides = (
98                  _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
99              )
100
101         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
102             out_root = spec.out_uri.rstrip("/")
103             token = self._token or uuid.uuid4().hex
104             done_uri = f"{out_root}/_dispatch/{token}.done"
105             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
106                                "--done-uri", done_uri]
107             if spec.segmenter:
108                 argv += ["--segmenter", spec.segmenter]
109             for kv in (*spec.config_overrides, *self._container_overrides):
110                 argv += ["--set", kv]
111
112             execution = self._client.run_job(self._job_name, argv, region=self._region,
113                                              project=self._project)
114             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
115                      self._job_name, execution, done_uri)
116
117             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
118                                 label="cloud-run-infer", logger=LOG)
119             video_id = str(marker.get("video_id", ""))
120             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
121             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
122             rc = int(marker.get("returncode", 0))
123             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
124             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
125
126         # --- storage-backed completion signal (bucket-poll) ---
127         def _read_marker(self, done_uri: str) -> Optional[dict]:
128             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
129             ref = parse_uri(done_uri)
130             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
131                 return None
132             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
133
134         def _read_json(self, uri: str) -> dict:
135             try:
136                 local = fetch(uri, config=self._storage_config)
137                 with open(local, "r", encoding="utf-8") as f:
138                     data: Any = json.load(f)
139                 return data if isinstance(data, dict) else {}
140             except Exception:
141                 return {}
142
```

7. user (2026-06-21T10:45:32.326Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
```

```
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
```

```
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

8. user (2026-06-21T10:45:32.611Z)

```
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3       All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4       worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5       ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6       selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7       """
8       import json
9       import os
10
11      from backend.compute import ComputeJobSpec, get_compute_backend
12      from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
13      from backend.config import AppConfig
14      from backend.infrastructure.execution_policy import ExecutionPolicy
15
16      # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
17      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
18
19
20      class _FakeRunClient(CloudRunJobClient):
21          """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
22          exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
23
24          def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
25              self.calls: list[dict] = []
26              self.video_id, self.segment_count, self.returncode = video_id, segment_count,
returncode
27
28          def run_job(self, job_name, argv, *, region, project=None):
29              self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
30              out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
31              done_uri = argv[argv.index("--done-uri") + 1]
32              outputs = os.path.join(out_uri, "outputs", self.video_id)
33              os.makedirs(outputs, exist_ok=True)
34              with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
35                  json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
36                             "status": "success"}, f)
37              os.makedirs(os.path.dirname(done_uri), exist_ok=True)
38              with open(done_uri, "w", encoding="utf-8") as f:
39                  json.dump({"video_id": self.video_id, "returncode": self.returncode,
40                             "segment_count": self.segment_count, "status": "success"}, f)
41              return "exec-123"
42
43
44      # ---------------------------------------------------------------- factory (default-
OFF)
45
46      def _cfg(compute_target="", storage_backend="local"):
```

```
        return AppConfig.model_validate({
            "deployment": {"compute_target": compute_target},
            "storage": {"backend": storage_backend},
        })


    def test_factory_is_none_for_local_targets():
        """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
        assert get_compute_backend(_cfg("")) is None
        assert get_compute_backend(_cfg("local_gpu")) is None
        assert get_compute_backend(_cfg("cpu_mock")) is None


    def test_factory_returns_cloud_run_for_cloud_target():
        backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
        assert isinstance(backend, CloudRunCompute)


    # ------------------------------------------------------------ CloudRunCompute dispatch
+ poll

    def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
        client = _FakeRunClient(video_id="abc123", segment_count=3)
        backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-
south1",
                                  project="proj", storage_config={}, policy=_FAST,
token="tok")
        out = str(tmp_path / "bucket")
        result = backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))

        assert result.returncode == 0
        assert result.video_id == "abc123"
        assert result.outputs_uri == f"{out}/outputs/abc123/"
        assert result.report.get("segment_count") == 3
        # exactly one Cloud Run Job execution was triggered, in the right region/project
        assert len(client.calls) == 1
        assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] ==
"asia-south1"


    def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
        """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
        native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST,
token="tok")
        backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
            config_overrides=("sport=tennis",)))

        argv = client.calls[0]["argv"]
        assert argv[0] == "infer"
        assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        # the container-inline overrides come last (win) and the user override is carried
through
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined
        assert "indexing.detector_impl=native" in joined
        assert "sport=tennis" in joined
        # user override precedes the inline-forcing overrides in --set order
```

```
        assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")


    def test_unique_token_per_call_when_not_injected(tmp_path):
        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
        assert done1 != done2


    # ------------------------------------------------- worker wiring: dispatch branch +
done-marker

    def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
        """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
        from backend.compute.base import ComputeResult
        from backend.worker import __main__ as wmain

        seen = {}

        class _FakeBackend:
            def run_infer(self, spec):
                seen["spec"] = spec
                return ComputeResult(returncode=0, video_id="vidX",
outputs_uri="gs://b/outputs/vidX/")

        monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
        cfg = tmp_path / "c.json"
        cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gcs"}}')
        rc = wmain.cmd_infer(wmain.build_parser().parse_args([
            "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
            "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
        assert rc == 0
        assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
        assert seen["spec"].segmenter == "wasb_hybrid"


    def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
        """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
        remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
        from backend.worker import __main__ as wmain

        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
                return {"validator_name": "fake", "passed": True, "message": "ok"}

        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_OK()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
```

```
158
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
159                             lambda p: 30.0)
160
161         cfg = tmp_path / "c.json"
162         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
163                                    "output": {"output_dir": str(tmp_path / "o")}}))
164         video = tmp_path / "in.mp4"
165         video.write_bytes(b"FAKEVIDEO")
166         out = tmp_path / "bucket"
167         done = str(out / "_dispatch" / "tok.done")
168
169         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
170             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
171             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
done]))
172         assert rc == 0
173         marker = json.loads(open(done, encoding="utf-8").read())
174         assert marker["returncode"] == 0 and marker["status"] == "success"
175         assert marker["video_id"] and marker["segment_count"] >= 1
176
177
178     def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
179         """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
180         from backend.worker import __main__ as wmain
181
182         class _OK:
183             passed = True
184             validator_name = "fake"
185             message = "ok"
186
187             def model_dump(self):
188                 return {"validator_name": "fake", "passed": True, "message": "ok"}
189
190         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
191                             lambda path, cfg: ([_OK()], {}))
192         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
193
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
194                             lambda p: 30.0)
195         cfg = tmp_path / "c.json"
196         cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
197                                    "output": {"output_dir": str(tmp_path / "o")}}))
198         video = tmp_path / "in.mp4"
199         video.write_bytes(b"FAKEVIDEO")
200         out = tmp_path / "bucket"
201         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
202             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
203             "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
204         assert rc == 0
205         assert not (out / "_dispatch").exists()  # no marker dir when --done-uri is omitted
206
207
208     # ------------------------------------------------------------ Dockerfile structural guard
(no build)
209
210     def test_dockerfile_has_required_directives():
211         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
212         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
worker entrypoint."""
213         from pathlib import Path
```

```
214
215         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
216         assert "FROM nvidia/cuda" in text                       # GPU base (CUDA)
217         assert "ffmpeg" in text                                  # stitch/decode
218         assert "fonts-dejavu" in text                            # the watermark filter font
219         assert "torch==1.11.0+cu113" in text                    # the verified WASB stack
220         assert "WASB-SBDT" in text                               # the shuttle detector repo
221         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
222         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
223
```

9. assistant (2026-06-21T10:45:35.008Z)

Now let me read the worker `__main__.py` to understand the dispatch branch and `_write_done_marker`, plus the execution_policy `poll_until`.

10. user (2026-06-21T10:45:36.356Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import sys
24      import tempfile
25      from typing import Any, List
26
27      from backend import storage
28      from backend.compute import ComputeJobSpec, get_compute_backend
29      from backend.config import (
30          StartupCheckError,
31          enforce_startup_checks,
32          load_config,
33          probe_cuda_available,
34      )
35      from backend.infrastructure.database import Database
36      from backend.pipeline.segmenters.base import segmenter_registry
37      from backend.pipeline.validators.base import registry as validator_registry
38      from backend.reporting import emit_indexer_report
39      from backend.results import Failure
40      from backend.utils.hashing import compute_video_id
41
42      logger = logging.getLogger("backend.worker")
43
```

```
44
45      def _coerce(v: str) -> Any:
46          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
47          low = v.lower()
48          if low in ("true", "false"):
49              return low == "true"
50          for cast in (int, float):
51              try:
52                  return cast(v)
53              except ValueError:
54                  pass
55          return v
56
57
58      def _apply_overrides(config: Any, sets: List[str]) -> None:
59          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60          for kv in sets or []:
61              key, sep, val = kv.partition("=")
62              if not sep:
63                  raise ValueError(f"--set expects k=v, got {kv!r}")
64              parts = key.split(".")
65              obj = config
66              for p in parts[:-1]:
67                  obj = getattr(obj, p)
68              setattr(obj, parts[-1], _coerce(val))
69
70
71      def _write_intervals_csv(segments: List[dict], path: str) -> None:
72          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73          artifact (same schema as the rally-annotator labels)."""
74          with open(path, "w", newline="") as f:
75              w = csv.writer(f)
76              w.writerow(["start", "end", "shot_count", "ending_reason"])
77              for s in segments:
78                  w.writerow([
79                      f'{float(s.get("start_time", 0.0)):.3f}',
80                      f'{float(s.get("end_time", 0.0)):.3f}',
81                      s.get("shot_count", ""),
82                      s.get("ending_reason", s.get("shot_count_confidence", "")),
83                  ])
84
85
86      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87          with open(path, "w") as f:
88              json.dump({
89                  "video_id": video_id,
90                  "status": status,
91                  "segment_count": len(segments),
92                  "segments": [{"start": float(s.get("start_time", 0.0)),
93                                "end": float(s.get("end_time", 0.0))} for s in segments],
94              }, f, indent=2)
95
96
97      def _run_startup_checks(config: Any) -> bool:
98          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99          best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
```

```
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False  # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch."""
120         spec = ComputeJobSpec(
121             video_uri=args.video_uri, out_uri=args.out_uri,
122             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
123         )
124         result = compute.run_infer(spec)
125         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
126                     result.video_id, result.returncode, result.outputs_uri)
127         return result.returncode
128
129
130     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
131                            status: str, storage_cfg: dict, scratch: str) -> None:
132         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
133         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
134         try:
135             marker = {"video_id": video_id, "returncode": returncode,
136                       "segment_count": segment_count, "status": status}
137             local = os.path.join(scratch, "_done.json")
138             with open(local, "w", encoding="utf-8") as f:
139                 json.dump(marker, f)
140             storage.put(local, done_uri, config=storage_cfg)
141             logger.info("[worker] wrote completion marker -> %s", done_uri)
142         except Exception as e:  # never fail a good run because the marker push hiccuped
143             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
144
145
146     def cmd_infer(args: argparse.Namespace) -> int:
147         config = load_config(args.config) if args.config else load_config()
148         _apply_overrides(config, args.set)
149         if not _run_startup_checks(config):
150             return 2
151
152         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
153         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
154         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
```

```
(the
155         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
156         compute = get_compute_backend(config)
157         if compute is not None:
158             return _dispatch_remote(compute, args)
159
160         storage_cfg = config.storage.model_dump()
161
162         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
163         os.makedirs(scratch, exist_ok=True)
164         config.output.output_dir = scratch
165
166         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
167         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
168         print(f"[worker] pulled {args.video_uri} -> {local_video}")
169
170         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
171         video_id = compute_video_id(local_video)
172
173         # 3. VALIDATE (same registry as the main CLI).
174         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
175         failures = [r for r in val_results if not r.passed]
176         db = Database(os.path.join(scratch, config.output.db_name))
177         db.add_video(video_id, local_video, "failed" if failures else "processed",
178                      [r.model_dump() for r in val_results])
179
180         segments: List[dict] = []
181         segmenter_actual = None
182         segmentation_ran = False
183         status = "failed"
184         try:
185             if failures:
186                 print("[worker] validation FAILED: "
187                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
188                 return 2
189             seg_name = args.segmenter or config.indexing.default_segmenter
190             segmenter_cls = segmenter_registry.get(seg_name)
191             if not segmenter_cls:
192                 print(f"[worker] unknown segmenter: {seg_name!r} "
193                       f"(have {list(segmenter_registry.list_available())})")
194                 return 2
195             segmenter_actual = seg_name
196             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
197             segmentation_ran = True
198             if isinstance(result, Failure):
199                 print(f"[worker] segmenter FAILED: {result.message}")
200                 return 3
201             segments = result.value if hasattr(result, "value") else result
202             status = "success"
203             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
204             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
205             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
206             if not segments:
207                 detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
208                                  or getattr(config.indexing, "detector_impl", None) or
"auto")
```

```
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
                    "the WASB env produced no detections, or the input resolution differs
from the tuned "
                    "proxy resolution. Re-run with -v for the native command + frame
counts.",
                    video_id, segmenter_actual, detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
        # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
        # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
        if getattr(args, "done_uri", None):
            _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
```

```
259
260         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
261         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
262         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
263         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
264         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
265         rather than guesses.
266         """
267         import json
268         import subprocess
269         try:
270             out = subprocess.check_output(
271                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
272                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
273                 stderr=subprocess.DEVNULL).decode()
274             st = (json.loads(out).get("streams") or [{}])[0]
275             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
276             fps = float(num) / float(den or "1")
277             width = float(st.get("width") or 0)
278             if fps > 0 and width > 0:
279                 return fps, width
280         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
281             pass
282         return None
283
284
285     def _resolve_train_detector(args: argparse.Namespace, config: Any):
286         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
287         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
288         stub=CPU mock). Returns the runner, or prints an error + returns None.
289
290         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
291         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
292         no shuttle detector and is rejected.
293         """
294         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
295
296         seg_cls = segmenter_registry.get(args.segmenter)
297         if not seg_cls:
298             print(f"[worker] unknown segmenter: {args.segmenter!r} "
299                   f"(have {list(segmenter_registry.list_available())})")
300             return None
301         seg = seg_cls(None, config)
302         if not isinstance(seg, TrajectoryHybridSegmenter):
303             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
304                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
305             return None
306         try:
307             runner, _ = seg._resolve_runner(config.get("indexing", {}))
308         except NotImplementedError as e:
309             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
310             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
311             print(f"[worker] detector not available: {e}")
312             return None
```

```
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

```

```
        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
```

```
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        p.add_argument("-v", "--verbose", action="store_true",
                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--done-uri", default=None,
                        help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
                             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
                             "Default-OFF: omit it for the normal local run.")
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
                        help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
                             "(needs videos/ in the corpus; honours detector_impl).")
        pt.add_argument("--segmenter", default="wasb_hybrid",
```

```
471                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
472         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
473         pt.add_argument("--config", default=None)
474         pt.add_argument("--scratch", default=None)
475         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
476         pt.add_argument("--frame-width", type=float, default=1280.0,
477                         help="synth trajectory frame width (detector probes the video)")
478         pt.add_argument("--set", action="append", default=[], metavar="k=v")
479         pt.set_defaults(func=cmd_train)
480         return p
481
482
483     def _setup_logging(verbose: bool) -> None:
484         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
485         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
486         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
487         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
488         entry), not at import, so importing the module never reconfigures a caller's
logging."""
489         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
490         # configured logging at import), which would leave the level at WARNING and re-
suppress the
491         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
492         logging.basicConfig(
493             level=logging.DEBUG if verbose else logging.INFO,
494             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
495             datefmt="%H:%M:%S",
496             force=True,
497         )
498
499
500     def main(argv=None) -> int:
501         args = build_parser().parse_args(argv)
502         _setup_logging(getattr(args, "verbose", False))
503         return args.func(args)
504
505
506     if __name__ == "__main__":
507         sys.exit(main())
508
```

11. user (2026-06-21T10:45:36.886Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
```

```
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13      #
14      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20      ENV DEBIAN_FRONTEND=noninteractive \
21          PYTHONUNBUFFERED=1 \
22          PIP_NO_CACHE_DIR=1 \
23          CONDA_DIR=/opt/conda \
24          WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27      RUN apt-get update && apt-get install -y --no-install-recommends \
28              ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29              python3 python3-pip python3-venv \
30          && rm -rf /var/lib/apt/lists/*
31
32      # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33      RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34          && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35          && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36          && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37          && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40              torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41          && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43              hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45      # --- Main application env ---
46      WORKDIR /app
47      COPY pyproject.toml README.md ./
48      COPY backend ./backend
49      COPY training ./training
50      RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55          WASB_REPO_DIR=/opt/models/WASB-SBDT \
56          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57          RALLY_DETECTOR_IMPL=native
58
```

```
59      # The dispatcher (CloudRunCompute) supplies the per-execution args:
60      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
61      ENTRYPOINT ["python3", "-m", "backend.worker"]
62
```

12. user (2026-06-21T10:45:36.977Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8       > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9       > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11      ## 0. Prerequisites (owner)
12      - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13      - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14        `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15      - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17      ## 1. Build + push the image (Artifact Registry)
18      ```bash
19      PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20      gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21      gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22      ```
23      *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25      ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26      ```bash
27      gcloud run jobs create rally-worker \
28        --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29        --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30        --max-retries 0 --task-timeout 3600 \
31        --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32      # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33      ```
34
35      ## 3. Wire the dispatcher (control plane)
36      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38      ```bash
39      export DEPLOYMENT_PROFILE=cloud-serving
40      export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42      # running in-process; the container runs the SAME worker INLINE (compute_target forced
```

```
local_gpu).
43      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44      ```
45
46      ## 4. M7 acceptance (the owner runlog)
47      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52      ## Gotchas
53      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times
out
58        (`ExecutionPolicy.max_wait_sec`). A failure-marker is a follow-up.
59      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
60
```

13. assistant (2026-06-21T10:45:40.253Z)

Now let me examine the `poll_until` implementation and `ExecutionPolicy` to understand the wait-path behavior, which is central to lens question (1).

14. user (2026-06-21T10:45:40.995Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17      from __future__ import annotations
18
19      import logging
20      import threading
21      import time as _time
22      from dataclasses import asdict, dataclass
```

```
    from enum import Enum
    from typing import Callable, Optional, TypeVar

    T = TypeVar("T")

    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
    LOG = logging.getLogger("execution.policy")


    class WaitMode(str, Enum):
        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
        WAIT_AND_RESUME = "wait_and_resume"   # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
        ABORT = "abort"                       # one attempt; on hang/fail, stop immediately
        BLOCK = "block"                       # bounded in-process blocking wait (no
reschedule)


    class PolicyTimeout(TimeoutError):
        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""


    @dataclass(frozen=True)
    class ExecutionPolicy:
        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
        mode: WaitMode = WaitMode.WAIT_AND_RESUME
        poll_every_n_sec: float = 10.0
        op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer
that was missing
        max_wait_sec: float = 1800.0          # total wall budget for a poll loop
        max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
        backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
        on_timeout: str = "reschedule"        # reschedule | partial | fail  (consumed by
P2/P3)
        resumable: bool = True

        def to_dict(self) -> dict:
            d = asdict(self)
            d["mode"] = self.mode.value
            return d

        @classmethod
        def from_dict(cls, d: dict) -> "ExecutionPolicy":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            kw = {k: v for k, v in d.items() if k in known}
            if "mode" in kw and not isinstance(kw["mode"], WaitMode):
                kw["mode"] = WaitMode(kw["mode"])
            return cls(**kw)


    #: The sane default (also what a job gets if it declares no policy).
    DEFAULT_POLICY = ExecutionPolicy()


    def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
        """Deterministic exponential backoff (no jitter ? reproducible tests)."""
        return policy.backoff_base_sec * (2 ** attempt)


    def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
```

```
                    label: str = "op", logger: Optional[logging.Logger] = None,
                    sleep: Callable[[float], None] = _time.sleep) -> T:
        """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.

        Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
        last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
            # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except BaseException as e:        # relay ANY error (incl. timeouts) to the
caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
                else:
                    if attempt:
                        log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
                    return box["val"]               # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
                log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
                            policy.op_timeout_sec)
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
                   label: str = "op", logger: Optional[logging.Logger] = None,
                   sleep: Callable[[float], None] = _time.sleep,
                   monotonic: Callable[[], float] = _time.monotonic) -> T:
        """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
        ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
```

```
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                  label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151
```

15. assistant (2026-06-21T10:45:45.461Z)

Now I have a complete picture. Let me verify the lens question (1) claim: does any test exercise `poll_until`'s wait path where the marker appears only after >=1 None poll? Let me check the storage layer's `exists`/`fetch`/`put` to confirm the fake's marker is written synchronously and that the bucket-poll path is real. Let me also confirm the config validation accepts `compute_target`.

16. user (2026-06-21T10:45:47.101Z)

```
backend/config/loader.py:167:            ct = merged["deployment"].get("compute_target")
backend/config/loader.py:170:                # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend/config/loader.py:172:                # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend/config/loader.py:173:                merged["deployment"]["compute_target"] = canonical
backend/config/loader.py:176:                    "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend/config/loader.py:180:            logger.info("[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend/config/loader.py:181:                        profile,
merged["deployment"].get("compute_target"))
backend/config/models.py:351:    compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend/config/presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend/config/presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend/config/presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend/config/presets.py:45:        "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend/config/presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend/config/presets.py:58:        "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
Binary file backend/config/__pycache__/loader.cpython-313.pyc matches
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
Binary file backend/config/__pycache__/presets.cpython-313.pyc matches
```

17. user (2026-06-21T10:45:48.474Z)

```
backend\storage\__init__.py:94:def get_storage(backend: Optional[str] = None, *,
backend\storage\__init__.py:106:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None,
*,
backend\storage\__init__.py:114:def put(local_path: "str | os.PathLike[str]", uri: str, *,
backend\storage\gdrive.py:112:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py:118:    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
backend\storage\gdrive.py:122:    def exists(self, ref) -> bool:
backend\storage\s3.py:38:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py:48:    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
backend\storage\s3.py:53:    def exists(self, ref) -> bool:
backend\storage\ref.py:48:def parse_uri(uri: str) -> StorageRef:
backend\storage\base.py:22:    def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend\storage\base.py:28:    def put(self, local_path: "str | os.PathLike[str]", ref:
StorageRef, *,
backend\storage\base.py:33:    def exists(self, ref: StorageRef) -> bool: ...
backend\storage\gcs.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:41:    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
backend\storage\gcs.py:45:    def exists(self, ref) -> bool:
backend\storage\local.py:29:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\local.py:42:    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
backend\storage\local.py:49:    def exists(self, ref) -> bool:
```

18. assistant (2026-06-21T10:45:51.603Z)

Now let me run the test file to confirm it passes and understand the actual execution, and verify the lens-1 claim concretely about whether `poll_until`'s wait path is exercised.

19. user (2026-06-21T10:46:01.426Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 9 items

tests/test_compute_backend.py::test_factory_is_none_for_local_targets PASSED [ 11%]
tests/test_compute_backend.py::test_factory_returns_cloud_run_for_cloud_target PASSED [ 22%]
tests/test_compute_backend.py::test_cloud_run_dispatch_polls_and_returns_result PASSED [ 33%]
tests/test_compute_backend.py::test_dispatch_argv_forces_inline_and_carries_overrides PASSED [
44%]
tests/test_compute_backend.py::test_unique_token_per_call_when_not_injected PASSED [ 55%]
tests/test_compute_backend.py::test_cmd_infer_dispatches_when_cloud_target PASSED [ 66%]
tests/test_compute_backend.py::test_worker_writes_done_marker_on_success PASSED [ 77%]
tests/test_compute_backend.py::test_worker_no_done_uri_is_unchanged PASSED [ 88%]
tests/test_compute_backend.py::test_dockerfile_has_required_directives PASSED [100%]

============================== 9 passed in 6.53s ==============================
```

20. assistant (2026-06-21T10:46:07.279Z)

All 9 pass. Now let me carefully analyze each lens question against the actual code.

Lens (1) ? poll_until wait path. The fake `_FakeRunClient.run_job` writes the marker synchronously (lines 28-41) before returning. So when `CloudRunCompute.run_infer` calls `poll_until` (line 117), the very first `check()` ? `_read_marker` finds the marker already present ? returns immediately. In `poll_until` (execution_policy.py line 140-143), `result` is non-None on the first iteration, so the loop returns without ever hitting `sleep` (line 150) or the `None` branch. Confirmed: **no M6 test exercises the wait path** (marker appears after ?1 None poll). Let me verify there's no other test in the repo that drives a delayed marker for CloudRunCompute.

21. user (2026-06-21T10:46:08.232Z)

```
tests/test_compute_backend.py
tests/test_execution_policy.py
tests/__pycache__/test_compute_backend.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_execution_policy.cpython-313-pytest-9.0.3.pyc
```

22. user (2026-06-21T10:46:11.633Z)

```
3:Every branch of run_bounded + poll_until + the ExecutionPolicy (de)serialisation, with
injected
17:    poll_until,
25:    """Deterministic clock: sleep() just advances monotonic() ? zero real waiting."""
29:        self.sleeps = []
34:    def sleep(self, s):
35:        self.sleeps.append(s)
41:def test_policy_defaults_and_mode_enum():
48:def test_policy_roundtrip_and_unknown_keys_ignored():
59:def test_run_bounded_success_first_try():
61:    assert run_bounded(lambda: 42, DEFAULT_POLICY, label="ok", sleep=clock.sleep,
logger=LOGGER) == 42
62:    assert clock.sleeps == []                                  # no retry, no backoff
65:def test_run_bounded_retries_failure_then_succeeds():
76:    assert run_bounded(flaky, pol, label="flaky", sleep=clock.sleep, logger=LOGGER) == "ok"
78:    assert clock.sleeps == [1.0, 2.0]                           # exponential backoff between
the 2 failures
81:def test_run_bounded_exhausts_and_raises_last_exception():
89:        run_bounded(always_fail, pol, label="bad", sleep=clock.sleep, logger=LOGGER)
90:    assert len(clock.sleeps) == 2                               # 3 attempts -> 2 backoffs
93:def test_run_bounded_hang_raises_policy_timeout():
97:        run_bounded(lambda: time.sleep(2.0), pol, label="hang", sleep=clock.sleep,
logger=LOGGER)
100:def test_run_bounded_hang_then_success():
107:            time.sleep(2.0)                                    # hangs past op_timeout on
attempt 1
111:    assert run_bounded(hang_once, pol, label="h", sleep=clock.sleep, logger=LOGGER) ==
"recovered"
115:def test_run_bounded_abort_mode_is_single_attempt():
125:        run_bounded(fail, pol, label="abort", sleep=clock.sleep, logger=LOGGER)
126:    assert calls["n"] == 1 and clock.sleeps == []           # ABORT -> no retries
129:# ------------------------------------------------------------- poll_until
131:def test_poll_until_ready_immediately():
133:    assert poll_until(lambda: "done", DEFAULT_POLICY, label="p",
134:                      sleep=clock.sleep, monotonic=clock.monotonic, logger=LOGGER) == "done"
135:    assert clock.sleeps == []                                  # ready on the first check
138:def test_poll_until_ready_after_n_polls():
144:        return "value" if state["n"] >= 4 else None
147:    assert poll_until(check, pol, label="p", sleep=clock.sleep, monotonic=clock.monotonic,
149:    assert state["n"] == 4 and clock.sleeps == [10.0, 10.0, 10.0]
152:def test_poll_until_times_out():
156:        poll_until(lambda: None, pol, label="p", sleep=clock.sleep,
161:def test_poll_logging_is_debug_not_info(caplog):
167:        return "v" if state["n"] >= 3 else None
171:        poll_until(check, pol, label="job7", sleep=clock.sleep,
```

23. assistant (2026-06-21T10:46:28.348Z)

This confirms: `poll_until`'s wait/loop path (None then ready) is covered by `test_poll_until_ready_after_n_polls` in `test_execution_policy.py` (the policy's own unit), but **not** through `CloudRunCompute`. The M6 tests only ever hit the immediate-return path. This is the gap the lens flags.

Now let me analyze the remaining lens questions carefully.

Lens (2) ? does `test_cloud_run_dispatch_polls_and_returns_result` prove the right (video_id-keyed) output is read vs coincidence? The fake writes `report.json` to `outputs/abc123/` and the marker carries `video_id=abc123`. The test asserts `outputs_uri == f"{out}/outputs/abc123/"` and `report.segment_count == 3`. The marker's `video_id` drives `outputs_uri` (cloud_run.py line 120). But here's the key: the fake writes the report's `segment_count` (3) into BOTH `report.json` and the marker. The `ComputeResult.report` comes from `report.json` (line 121), but the marker ALSO contains `segment_count` (fake line 40). Let me check whether the assertion `result.report.get("segment_count") == 3` could pass even if `_read_json(report.json)` returned `{}` ? it can't, because `report` defaults to `{}` and `.get` would return None ? 3. So reading report.json IS load-bearing. But does it prove the path is *video_id-keyed* rather than coincidental? The fake only ever writes ONE outputs dir (`outputs/abc123/`); if `outputs_uri` were mis-constructed, `_read_json` would silently return `{}` (the except swallows it), and `segment_count` would be None. So the test does discriminate on the path. Let me verify there's no other `outputs/` dir written. Confirmed from the fake: only `outputs/<video_id>`. So a wrong path ? empty report ? assertion fails. This is reasonably grounded, not pure coincidence.

However, there's a subtle weakness: `video_id="abc123"` could be read from EITHER the marker OR report.json. The assertion `result.video_id == "abc123"` only proves the marker's video_id is propagated. That's fine.

Lens (3) ? `test_worker_writes_done_marker_on_success` non-vacuous? Line 175: `assert marker["video_id"] and marker["segment_count"] >= 1`. The stub detector must produce ?1 segment. Let me confirm the stub actually yields segments so this guard is real and not trivially satisfied. Let me check the stub path.

24. user (2026-06-21T10:46:29.810Z)

```
62:            f"detector_impl='native' is not supported for the "
64:            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
67:        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.
69:        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
70:        ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
77:        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78:        if impl == "stub":
79:            from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
80:            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
82:            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
84:            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
138:    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
191:        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192:        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
197:            # e.g. detector_impl="native" for a family without a native runner (tracknet, or
```

25. assistant (2026-06-21T10:46:35.342Z)

The stub runner produces deterministic trajectories from the video (30s duration mocked). The test asserts `segment_count >= 1`, which is a genuine guard ? if the stub produced 0 segments, the assertion would fail. So the test is non-vacuous (it proves the marker reflects a real run with output). This is sound.

Now let me check the test for `--done-uri` marker writing more carefully: the marker is written by `_write_done_marker` which uses `storage.put(local, done_uri)`. In the test, `done` is a local path (`out / "_dispatch" / "tok.done"`). Let me verify `storage.put` with a local path creates the parent dir ? otherwise the test would be relying on something. The test passed, so

it works. Let me verify the local storage `put` behavior.

26. user (2026-06-21T10:46:35.979Z)

```
1       """Local-filesystem storage backend ? the default, zero new dependencies.
2
3       Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4       file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5       the whole pipeline behaves exactly as before the storage layer existed.
6       """
7       from __future__ import annotations
8
9       import os
10      import shutil
11      from datetime import datetime
12      from typing import Iterator
13
14      from backend.storage.base import StorageBackend
15      from backend.storage.ref import StorageRef, StorageStat
16
17
18      class LocalStorage(StorageBackend):
19          def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20              # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21              self.base_dir = base_dir
22
23          def _path(self, ref: StorageRef) -> str:
24              p = ref.key
25              if self.base_dir and not os.path.isabs(p):
26                  return os.path.join(self.base_dir, p)
27              return p
28
29          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30              # The object is already on the local FS ? return its path (identity). ``dst`` is
31              # honoured only if it names a *different* location (a copy is then made).
32              src = self._path(ref)
33              if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                  return src
35              dst = str(dst)
36              if os.path.exists(dst) and not overwrite:
37                  return dst
38              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39              shutil.copyfile(src, dst)
40              return dst
41
42          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43              dst = self._path(ref)
44              if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                  os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                  shutil.copyfile(str(local_path), dst)
47              return ref
48
49          def exists(self, ref) -> bool:
50              return os.path.exists(self._path(ref))
51
52          def stat(self, ref) -> StorageStat:
53              st = os.stat(self._path(ref))
54              return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56          def list(self, prefix) -> Iterator[StorageRef]:
57              base = self._path(prefix)
58              if os.path.isdir(base):
```

```
59                  for root, _dirs, files in os.walk(base):
60                      for fn in sorted(files):
61                          full = os.path.join(root, fn)
62                          yield StorageRef("local", "", full, f"local://{full}")
63              elif os.path.exists(base):
64                  yield StorageRef("local", "", base, f"local://{base}")
65
66          def delete(self, ref) -> None:
67              path = self._path(ref)
68              if os.path.exists(path):
69                  os.remove(path)
70
```

27. assistant (2026-06-21T10:47:00.048Z)

Good. Now let me carefully examine **Lens (5) ? any assertion that would pass even if the dispatch shim were broken?** Let me scrutinize `test_cmd_infer_dispatches_when_cloud_target` (lines 119-139). It monkeypatches `get_compute_backend` with a lambda returning `_FakeBackend()`, asserting the spec carries the right video_uri/segmenter. This proves `cmd_infer` calls `_dispatch_remote` and builds a correct spec. But note: it monkeypatches `wmain.get_compute_backend` to **always** return a backend regardless of config ? so this test does NOT verify that the real `get_compute_backend` actually returns non-None for `cloud-serving`. The config in the test sets `profile: cloud-serving` but since `get_compute_backend` is stubbed, the profile?compute_target wiring is irrelevant to this test. The factory wiring IS covered separately by `test_factory_returns_cloud_run_for_cloud_target` and `test_factory_is_none_for_local_targets`. So there's no false-confidence here ? coverage is split but complete.

One thing to check on lens (5): the monkeypatch signature. `wmain.get_compute_backend` is called as `get_compute_backend(config)` in cmd_infer (line 156) ? single positional arg. The monkeypatch lambda is `lambda config: _FakeBackend()`. That matches. Good.

Let me now verify the `--done-uri`/`_write_done_marker` returncode. Note line 248: `_write_done_marker(args.done_uri, video_id, 0, ...)` ? the returncode hardcoded to `0`. The marker is only written on the success path (after `return 2`/`return 3` early-returns). So `returncode` in the marker is always 0 when written. The test asserts `marker["returncode"] == 0`. That's consistent but let me note: the dispatcher reads `rc = int(marker.get("returncode", 0))` ? so a failed container writes no marker and the dispatcher times out (documented as a follow-up). Not a test gap per se, it's a known limitation. The test correctly only covers the success-marker case.

Now let me re-examine **lens (2)** more rigorously for a coincidence-passing concern. In `test_cloud_run_dispatch_polls_and_returns_result`, the marker carries `video_id=abc123` AND `segment_count=3`. The `report.json` also carries `segment_count=3`. The assertion `result.report.get("segment_count") == 3` reads from report.json. But could the test pass if `outputs_uri` were built from the WRONG key? Suppose a bug made `outputs_uri` empty ? then `report` would be `{}` (line 121 guards `if outputs_uri else {}`), and `.get("segment_count")` ? None ? 3 ? fails. Suppose `outputs_uri` were built but pointed to a nonexistent path ? `_read_json`'s `fetch` would raise, caught, return `{}` ? fails. So the path keying IS load-bearing for the report assertion. Good ? not coincidence.

But there's a subtle real gap worth flagging for lens (2): the `video_id` used to BUILD `outputs_uri` comes only from the **marker**, and the test's fake writes `report.json` to the exact same `video_id` directory. The test cannot distinguish "dispatcher correctly used marker.video_id to key the output path" from "dispatcher used some other source that happens to equal abc123" ? but since marker.video_id is the only abc123 source available to the dispatcher, this is fine. No real issue.

Let me double check there's no genuine assertion-too-weak in `test_dispatch_argv_forces_inline_and_carries_overrides`. It uses `" ".join(argv)` and substring `in` checks. A substring check like `"sport=tennis" in joined` is slightly loose (could match a larger token) but here the values are distinct and the ordering assertion (line 103) adds rigor. Acceptable.

Now let me confirm **lens (4) ? Dockerfile guard is string-match only.**

`test_dockerfile_has_required_directives` asserts specific substrings exist (`FROM nvidia/cuda`, `ffmpeg`, `fonts-dejavu`, `torch==1.11.0+cu113`, `WASB-SBDT`, the ENTRYPOINT, `WASB_PYTHON=`). This cannot catch: ordering errors, a broken multi-line RUN, the actual ENTRYPOINT being commented out (a `#` before it would still substring-match), or build failures. Let me verify the test could pass on a commented/broken Dockerfile to gauge whether this is a meaningful false-confidence point.

28. user (2026-06-21T10:47:01.384Z)

```
28:        ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
---
218:    assert "fonts-dejavu" in text                            # the watermark filter font
```

29. assistant (2026-06-21T10:47:15.659Z)

The Dockerfile installs `fonts-dejavu-core` and the test asserts substring `"fonts-dejavu"` ? matches. The string-match guard is inherently weak (a commented-out line still matches, can't validate the build), but the test docstring is honest about this: "CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives structurally." This is a deliberate, documented limitation, not false confidence ? it's the strongest feasible check without a CUDA build. Not a real finding to flag as a bug; it's an acknowledged trade-off.

Let me now verify one more thing for lens completeness: the watermark font. The Dockerfile comment says DejaVu is "the watermark filter font," but does the worker's stitch path actually run in this `infer` flow with the stub? Not relevant to test rigor. Let me instead confirm there isn't a hidden vacuous-pass in `test_worker_no_done_uri_is_unchanged` (the default-OFF guard). It asserts `not (out / "_dispatch").exists()`. This proves no marker dir is created without `--done-uri`. Solid negative-control. Good.

Now, the central lens-1 question: **is the untested wait path a meaningful gap for the M6 'bucket-poll' claim?** The README and module docstring prominently claim "bucket-poll" as the completion mechanism (the whole point of D16: scale-to-zero cold start absorbed by async poll). Yet every M6 test resolves the marker on the *first* `check()` ? the loop never iterates, never sleeps, never handles a transient `None`. The cold-start/async-wait scenario the design exists to handle is entirely unexercised through `CloudRunCompute`. The `_read_marker` "returns None while in progress" contract (the actual integration glue between the storage layer and `poll_until`) is never validated with a real None?present transition. `poll_until`'s own loop is tested in isolation, but the *composition* (`CloudRunCompute._read_marker` returning None, then a marker appearing) is not. This is a real, meaningful gap and is trivially closable with a fake whose `run_job` defers the marker write (e.g., writes it only on the Nth `_read_marker` call, or a fake storage that returns the marker after N polls).

Let me confirm there are no other tests elsewhere covering this transition through the compute backend before finalizing.

30. user (2026-06-21T10:47:16.781Z)

```
tests/test_compute_backend.py
```

31. assistant (2026-06-21T10:47:25.937Z)

Confirmed ? only `test_compute_backend.py` touches this, and as analyzed, it always resolves on the first poll.

I have a complete, grounded picture. Let me also do a final scan for an assertion that could pass with a broken shim in the dispatch-argv test ? specifically whether `--done-uri` presence is actually asserted to carry a *value* (not just the flag). Line 95: `assert "--done-uri" in argv` only checks the flag token exists, not that a value follows. If the shim appended `--done-uri` with no value, this test would still pass ? but the fake client immediately does `argv[argv.index("--done-uri") + 1]` (line 31) which would raise IndexError or grab the wrong token, so the broken case is caught elsewhere. And `test_cloud_run_dispatch_polls_and_returns_result` exercises the real done_uri value end-to-end. So no false confidence there.

Let me compile my findings. The analysis confirms one genuine, meaningful test-rigor gap (lens 1) and the lens-4 string-match concern is a documented, acceptable trade-off (not a real bug). Lenses 2, 3, 5 found no vacuous coverage. I'll report the one real finding plus a minor note on the wait-path being closable, and not invent nits.

32. user (2026-06-21T10:47:47.527Z)

Structured output provided successfully

33. assistant (2026-06-21T10:48:01.378Z)

I've completed the test-rigor / false-confidence review of the M6 change and returned findings via StructuredOutput.

Summary of what I found:

Major (1 real gap): The `poll_until` wait/loop path is never exercised through `CloudRunCompute`. The fake client (`tests/test_compute_backend.py:33-40`) writes the done-marker synchronously during dispatch, so every `CloudRunCompute` test resolves on `poll_until`'s very first `check()` ? the loop never iterates, never sleeps, never handles a transient `None`. The storage-backed `None`?present transition (the exact integration glue `_read_marker` adds at `backend/compute/cloud_run.py:127-132`, and the cold-start/scale-to-zero scenario the design exists for per D7/D16) has zero coverage through the compute backend. `execution_policy`'s own `test_poll_until_ready_after_n_polls` covers a delayed-ready check but with a hand-rolled counter, not `_read_marker` against the storage layer. Closable with a fake that defers the marker until the Nth poll.

Nit: The Dockerfile guard (`tests/test_compute_backend.py:210-223`) is substring-only and would pass on a commented-out/build-broken directive ? but this is a documented, acceptable trade-off (CI can't build a CUDA image; owner-verified on M7), flagged only so it isn't misread as proof the image builds.

On the other lens questions, I found no real issues to invent:
- **Lens 2** (`test_cloud_run_dispatch_polls_and_returns_result`): the video_id-keyed path IS load-bearing ? a mis-built `outputs_uri` makes `_read_json` silently return `{}`, so `report.get("segment_count") == 3` would fail. Not coincidence.
- **Lens 3** (`test_worker_writes_done_marker_on_success`): non-vacuous ? `marker["segment_count"] >= 1` is a genuine guard against a 0-segment stub run.
- **Lens 5**: no assertion passes with a broken shim. The `get_compute_backend` monkeypatch test splits factory-wiring coverage into the separate `test_factory_*` tests (which use the real factory), so the split is complete, not a hole.

All 9 tests pass locally (Python 3.13).

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\execution_policy.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_execution_policy.py`.